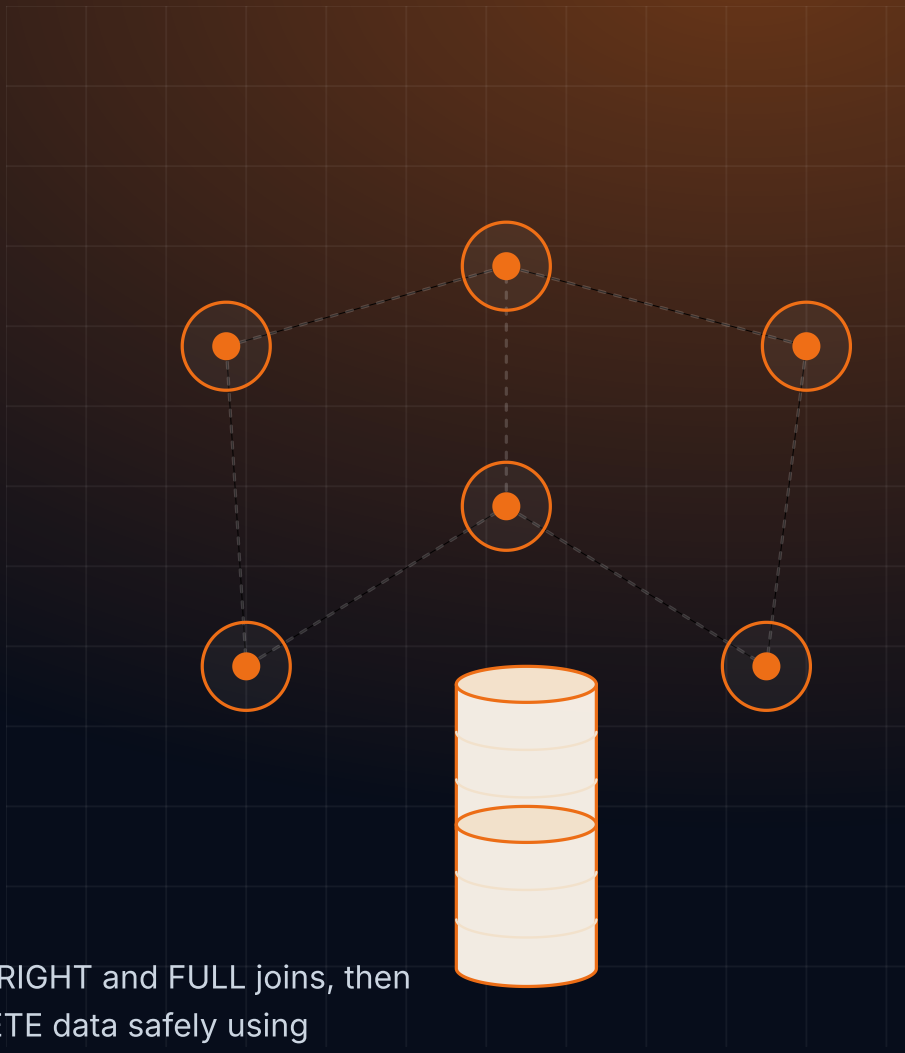


PHASE 04 / 07

Joins & DML

Combine tables with INNER, LEFT, RIGHT and FULL joins, then learn to INSERT, UPDATE and DELETE data safely using transactions.

SQL Joins | DML Commands

Real data lives in many tables. Joins put it back together; DML changes it. Together they are the heart of every load, merge and reconciliation job a data engineer writes.

CONTENTS

Inside this phase

SQL Joins 3

- Why we need joins 3
- INNER, LEFT, RIGHT and FULL: what happens to unmatched rows 4
- UNION, UNION ALL, INTERSECT and EXCEPT 7
- Finding what is missing: anti-joins 8
- CROSS JOIN and SELF JOIN 9
- Joining many tables: the star-schema query 10
- Four join traps every engineer falls into 12

DML Commands 14

- INSERT: adding rows 14
- Upsert: the idempotent load pattern 16
- UPDATE: changing existing rows 17
- DELETE: removing rows 18
- Transactions: all or nothing 19

Phase 4 Recap & Practice 21

- Cheat sheet 21
- Check yourself 21
- Practice lab 22
- Answer key 22

4.1 PART 4.1 · COMBINING TABLES

SQL Joins

Phase 2 taught you to split data into separate tables so each fact is stored once. A **join** is how you put the pieces back together at query time, matching rows from two tables on a shared key.

What you will learn

- Explain what a join does and write the `ON` condition correctly
- Use INNER, LEFT, RIGHT joins and emulate FULL join with UNION
- Find orphans and missing rows with anti-joins
- Use CROSS and SELF joins, and join many tables of a star schema
- Avoid the classic traps: ON vs WHERE, duplicate fan-out, NULL keys

Why it matters for data engineering. Joins are where pipelines quietly go wrong: a bad join key duplicates rows and inflates revenue, or drops rows and understates it. Row-count reconciliation before and after every join is a habit that separates reliable engineers from risky ones.

Why we need joins

The course uses two tiny tables. `orders` knows *which customer id* placed each order, but not the customer's name; `customers` knows names but not orders. A join links them through `orders.cust_id = customers.id`.

orders

o_id	cust_id	price
1	101	1000
2	201	1100
3	501	1200

cust_id = foreign key

customers

id	name	email
101	love	aa
201	ansh	bb
301	lamba	cc

id = primary key

JOIN

result of orders JOIN customers ON cust_id = id

o_id	price	id	name	email
1	1000	101	love	aa
2	1100	201	ansh	bb

Order 3 vanished (no customer 501) and customer 301 vanished (no orders): this is an INNER join.

Figure 1. A join matches rows by key and returns them side by side. What happens to unmatched rows is what distinguishes the join types.

SQL — Join syntax

```sql
SELECT   <columns from either table>
FROM     table_a  a
JOIN     table_b  b   ON a.key = b.key        -- the join condition
WHERE    …                                    -- filters apply AFTER the join
```

- **Alias** each table (`orders o`, `customers c`) and prefix every column (`o.price`). Short names make long queries readable and avoid *ambiguous column* errors.
- `ON` states how rows match. Most of the time it compares a **foreign key** to a **primary key**.
- `USING (col)` is shorthand when both tables use the same column name: `JOIN b USING (customer_key)`.

INNER, LEFT, RIGHT and FULL: what happens to unmatched rows

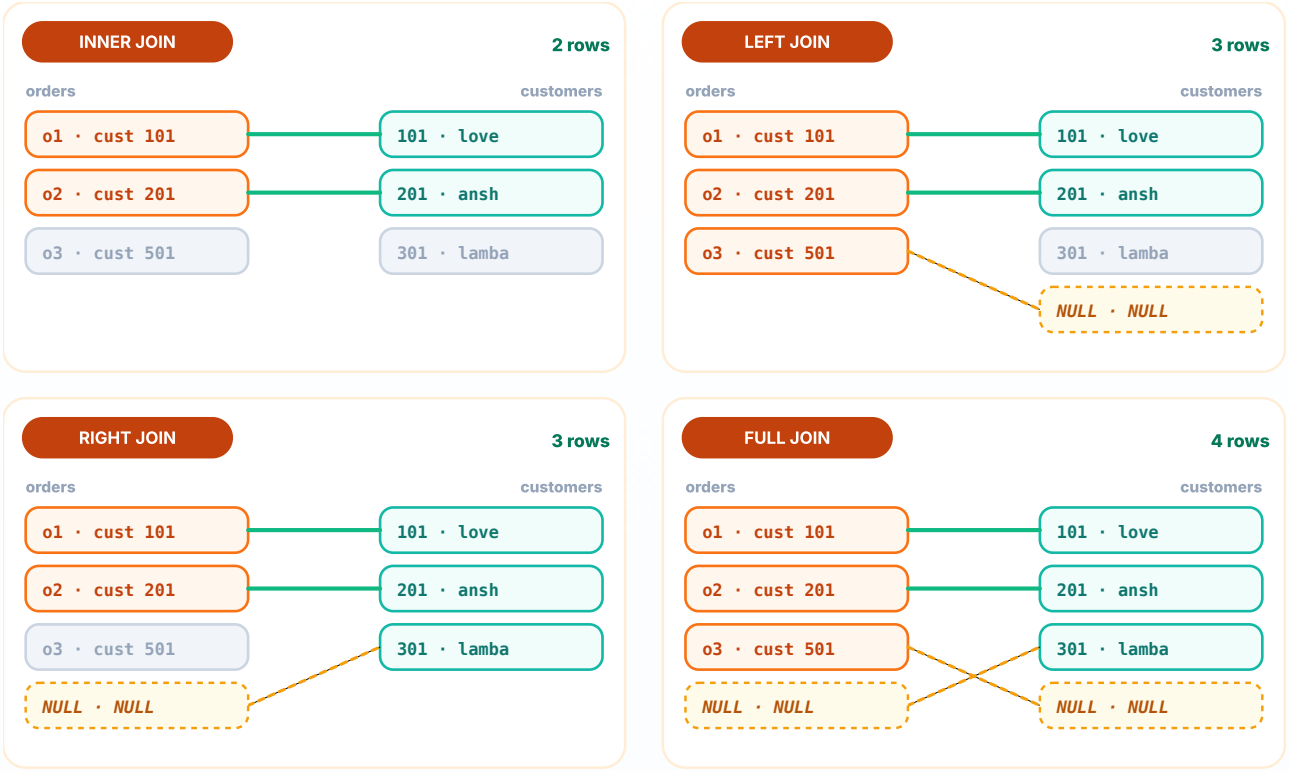
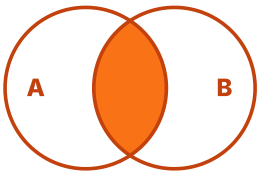


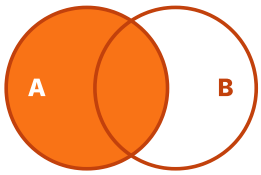
Figure 2. Green lines are matches. Dimmed boxes are rows that disappear; dashed amber lines are unmatched rows kept alive with NULLs on the other side.



INNER JOIN

```
FROM a
INNER JOIN b ON a.k = b.k
```

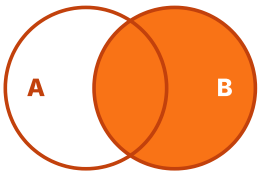
only rows with a match on **both** sides



LEFT JOIN

```
FROM a
LEFT JOIN b ON a.k = b.k
```

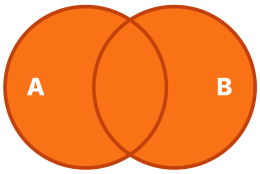
all of A, matching B (else NULL)



RIGHT JOIN

```
FROM a
RIGHT JOIN b ON a.k = b.k
```

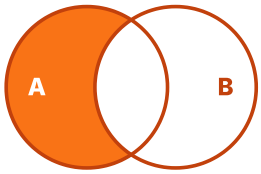
all of B, matching A (else NULL)



FULL JOIN

```
-- not in MySQL
-- LEFT UNION RIGHT
```

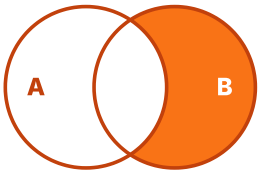
everything from both sides



LEFT ANTI

```
LEFT JOIN b ON …
WHERE b.k IS NULL
```

rows in A **with no** match in B



RIGHT ANTI

```
RIGHT JOIN b ON …
WHERE a.k IS NULL
```

rows in B **with no** match in A

INNER JOIN: only the matches

SQL — Course ch-4 · 2 rows: orders 1 and 2

```
SELECT *
FROM   orders o
INNER JOIN customers c
       ON o.cust_id = c.id;
```

RESULT — 2 ROWS

o_id	cust_id	price	id	name	email
1	101	1000	101	love	aa
2	201	1100	201	ansh	bb

LEFT JOIN: keep every row from the left table

SQL — Course ch-4 · 3 rows: order 3 survives, its customer columns are NULL

```sql
SELECT *
FROM   orders o
LEFT JOIN customers c
       ON o.cust_id = c.id;
```

RESULT — 3 ROWS

o_id	cust_id	price	id	name	email
1	101	1000	101	love	aa
2	201	1100	201	ansh	bb
3	501	1200	NULL	NULL	NULL

RIGHT JOIN: keep every row from the right table

SQL — Course ch-4 · 3 rows: customer 301 survives, order columns are NULL

```sql
SELECT *
FROM   orders o
RIGHT JOIN customers c
       ON o.cust_id = c.id;
```

RESULT — 3 ROWS

o_id	cust_id	price	id	name	email
1	101	1000	101	love	aa
2	201	1100	201	ansh	bb
NULL	NULL	NULL	301	lamba	cc

PRO TIP

A `RIGHT JOIN` is just a `LEFT JOIN` with the tables swapped. Most teams standardize on **LEFT JOIN only** (put the main table first), which keeps queries easy to read from top to bottom.

FULL JOIN: not supported, so use UNION

SQL — Course ch-4: MySQL has no FULL JOIN

```sql
SELECT * FROM orders o FULL JOIN customers c ON o.cust_id = c.id;
```

ERROR

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'FULL JOIN customers c ON o.cust_id = c.id' at line 1
```

SQL — Course ch-4 'UNIONS': LEFT + RIGHT = FULL · 4 rows

```sql
SELECT * FROM orders o LEFT  JOIN customers c ON o.cust_id = c.id
UNION
SELECT * FROM orders o RIGHT JOIN customers c ON o.cust_id = c.id;
```

RESULT — 4 ROWS

o_id	cust_id	price	id	name	email
1	101	1000	101	love	aa
2	201	1100	201	ansh	bb
3	501	1200	NULL	NULL	NULL
NULL	NULL	NULL	301	lamba	cc

UNION, UNION ALL, INTERSECT and EXCEPT

Joins add **columns** side by side. Set operators stack **rows** from two queries on top of each other. Both queries must return the same number of columns with compatible types; the column names come from the first query.

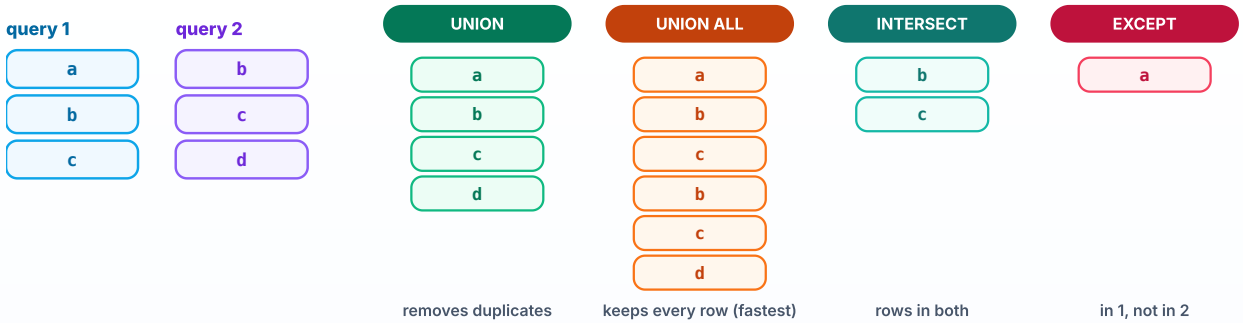


Figure 3. Set operators on two one-column queries.

SQL — UNION de-duplicates (and sorts in practice)

```sql
SELECT category FROM dim_product WHERE unit_price > 990 UNION SELECT category FROM dim_product WHERE
unit_price < 12 ORDER BY category;
```

RESULT — 2 ROWS

category
Sports
Toys

SQL — UNION ALL keeps duplicates

```sql
SELECT category FROM dim_product WHERE unit_price > 950 UNION ALL SELECT category FROM dim_product
WHERE unit_price > 970 ORDER BY category LIMIT 4;
```

RESULT — 4 ROWS

category
Books
Books
Books
Books

DATA-ENGINEERING LENS

Prefer `UNION ALL` unless you really need de-duplication: `UNION` must sort or hash every row to find duplicates. A classic pattern is stacking monthly partition tables (`sales_2024_01 UNION ALL sales_2024_02 …`) into one logical table. `INTERSECT` / `EXCEPT` need MySQL **8.0.31+**; on older versions use `IN` / `NOT EXISTS` .

Finding what is missing: anti-joins

A `LEFT JOIN` followed by `WHERE right_key IS NULL` returns left rows that have **no partner**. This *anti-join* is the standard way to detect orphans, missing loads and customers who never bought.

SQL — Orphan orders: customer 501 does not exist

```sql
SELECT o.*
FROM   orders o
LEFT JOIN customers c ON o.cust_id = c.id
WHERE  c.id IS NULL;
```

RESULT — 1 ROW

o_id	cust_id	price
3	501	1200

SQL — Customers who never ordered

```sql
SELECT c.*
FROM   customers c
LEFT JOIN orders o ON c.id = o.cust_id
WHERE  o.o_id IS NULL;
```

RESULT — 1 ROW

id	name	email
301	lamba	cc

SQL — Referential-integrity check on the real star schema: 0 means every sale has a product

```sql
SELECT COUNT(*) AS sales_without_product
FROM   fact_sales f
LEFT JOIN dim_product p ON f.product_key = p.product_key
WHERE  p.product_key IS NULL;
```

RESULT — 1 ROW

sales_without_product
0

DATA-ENGINEERING LENS

This exact query is a **data-quality test**. Warehouses do not enforce foreign keys, so teams run anti-joins (or dbt `relationships` tests) after every load to prove no fact row points at a missing dimension.

CROSS JOIN and SELF JOIN

CROSS JOIN: every combination

A cross join pairs **every row of A with every row of B** (the Cartesian product): 6 categories × 4 quarters = 24 rows. It is rarely the final answer but is perfect for building a *scaffold* of all combinations, which you then `LEFT JOIN` facts onto so that empty combinations show as zero instead of disappearing.

SQL — 6 categories × 4 quarters

```sql
SELECT c.category, q.quarter
FROM  (SELECT DISTINCT category FROM dim_product) c
CROSS JOIN
      (SELECT DISTINCT quarter  FROM dim_date)    q
ORDER BY c.category, q.quarter;
```

RESULT — 24 ROWS

category	quarter
Books	1
Books	2
Books	3
Books	4
Clothing	1

... 19 more rows not shown

WATCH OUT

Forgetting the `ON` clause (or writing `ON 1=1`, or joining on a non-key) silently creates a cross join. Two tables of 1 million rows would produce **a trillion rows**. Always sanity-check row counts.

SELF JOIN: a table joined to itself

When a table references itself, as an employee points to a manager who is also an employee, join it to itself using two different aliases.

SQL — Same table, two roles: e = the employee, m = their manager

```sql
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM   emp e
LEFT JOIN emp m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

RESULT — 5 ROWS

employee	manager
Asha	*NULL*
Ravi	Asha
Meena	Asha
Kabir	Asha
Nia	Kabir

Joining many tables: the star-schema query

Analytical queries start at the **fact table** and join out to each **dimension** they need. Each join is just another `JOIN … ON fact.fk = dim.pk`.

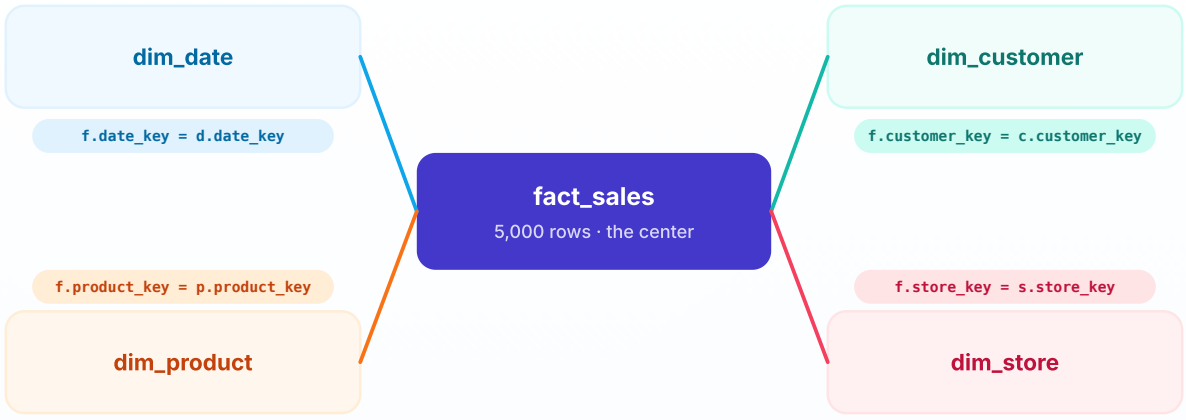


Figure 4. A star join: the fact table is joined to each dimension on the surrogate key.

SQL — Revenue by year and category: two joins, one aggregation

```sql
SELECT   d.year,
         p.category,
         ROUND(SUM(f.total_amount)) AS revenue
FROM     fact_sales  f
JOIN     dim_date    d ON f.date_key    = d.date_key
JOIN     dim_product p ON f.product_key = p.product_key
GROUP BY d.year, p.category
ORDER BY d.year, revenue DESC
LIMIT 6;
```

RESULT — 6 ROWS

year	category	revenue
2023	Clothing	326765
2023	Books	291461
2023	Electronics	264317
2023	Home & Kitchen	231839
2023	Sports	229594
2023	Toys	215351

SQL — Region × gender: two other dimensions

```sql
SELECT   s.region,
         c.gender,
         COUNT(*)                   AS sales,
         ROUND(SUM(f.total_amount)) AS revenue
FROM     fact_sales    f
JOIN     dim_store     s ON f.store_key    = s.store_key
JOIN     dim_customer  c ON f.customer_key = c.customer_key
GROUP BY s.region, c.gender
ORDER BY s.region, c.gender;
```

RESULT — 10 ROWS

region	gender	sales	revenue
Central	F	638	1807947
Central	M	664	1832066
East	F	460	1265184
East	M	442	1211747
North	F	396	1060086
North	M	394	1090023
South	F	505	1511084
South	M	480	1307755

... 2 more rows not shown

REMEMBER

`JOIN` alone means `INNER JOIN`. In a star schema every fact row has a valid dimension key, so inner joins keep all 5,000 rows. If some keys were missing, an inner join would silently drop those sales; use `LEFT JOIN` from the fact table when completeness matters.

Four join traps every engineer falls into

Trap 1 · Filtering in WHERE turns a LEFT JOIN into an INNER JOIN

SQL — Condition in ON: keeps all customers

```sql
SELECT c.name, o.o_id
FROM   customers c
LEFT JOIN orders o
       ON c.id = o.cust_id
      AND o.price > 1050;
```

RESULT 3 ROWS

name	o_id
ansh	2
love	NULL
lamba	NULL

SQL — Condition in WHERE: customers vanish

```sql
SELECT c.name, o.o_id
FROM   customers c
LEFT JOIN orders o ON c.id = o.cust_id
WHERE  o.price > 1050;
```

RESULT 1 ROW

name	o_id
ansh	2

In the second query, unmatched customers get `o.price = NULL`; `NULL > 1050` is not true, so `WHERE` throws them away: your LEFT JOIN quietly became an INNER JOIN. **Rule:** conditions on the *right-hand* table of a LEFT JOIN belong in `ON`.

Trap 2 · Fan-out: one-to-many joins duplicate the "one" side

Each department has one `budget`, but several employees. After joining, the budget repeats once per employee, and summing it **multiplies** the figure.

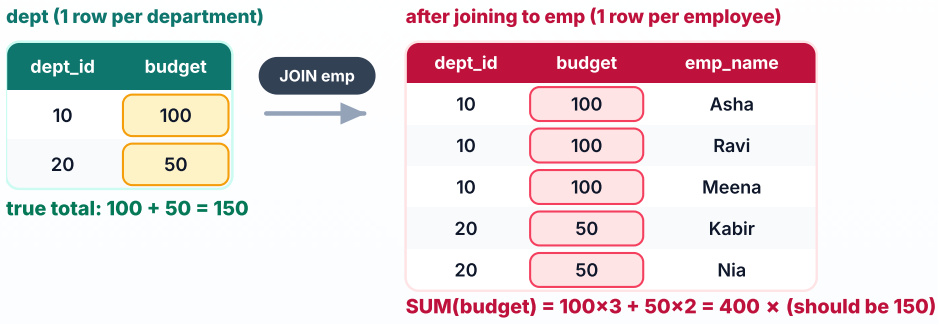


Figure 5. Join fan-out inflates parent-level measures. This is the #1 cause of revenue figures that are "mysteriously too high".

SQL — Wrong: budget is counted once per employee

```sql
SELECT SUM(d.budget) AS inflated_budget
FROM   dept d
JOIN   emp  e ON e.dept_id = d.dept_id;
```

RESULT — 1 ROW

inflated_budget
400

SQL — Right: aggregate before (or without) joining many-side rows

```sql
SELECT SUM(d.budget) AS correct_budget
FROM   dept d
WHERE  d.dept_id IN (SELECT dept_id FROM emp);
```

RESULT — 1 ROW

correct_budget
150

DATA-ENGINEERING LENS

Before trusting any join, count. Compare `COUNT(*)` of the left table with the joined result. If the joined count is bigger, the right side has duplicate keys; if smaller with a LEFT JOIN, something is wrong. Also check the key is unique: `SELECT key, COUNT(*) FROM t GROUP BY key HAVING COUNT(*) > 1`.

Trap 3 · NULL keys never match

`NULL = NULL` is not true, so rows whose join key is NULL are dropped by INNER joins and show no partner in LEFT joins. Clean or coalesce keys first, or deliberately use `ON a.k <=> b.k` (MySQL's NULL-safe equals).

Trap 4 · Mismatched key types or formats

Joining `INT` to `VARCHAR`, `'CUST0001'` to `'cust0001 '` (case/space differences), or `DATE` to `DATETIME` causes silent mismatches or full-table scans. Normalize both sides (`TRIM`, `UPPER`, `CAST`) in an earlier step.

INTERVIEW ANGLE

"Given tables A (100 rows) and B (80 rows), what is the minimum and maximum row count of A INNER JOIN B?" Min 0; max depends on duplicates in B: up to 100 × 80 = 8,000 if all keys are equal. "...and A LEFT JOIN B?" At least 100.

4.2 PART 4.2 · CHANGING DATA

DML Commands

DML (Data Manipulation Language) works on **rows**: INSERT adds them, UPDATE changes them, DELETE removes them. These statements permanently change data, so the habits you build here (preview first, use transactions) protect you in production.

What you will learn

- Insert one row, many rows, or the result of a query
- Update and delete exactly the rows you intend
- Use `ON DUPLICATE KEY UPDATE` for idempotent loads (upserts)
- Wrap changes in transactions and roll back mistakes
- Understand ACID and why InnoDB matters

Why it matters for data engineering. Every ELT job ends with DML: loading new rows, correcting old ones, removing stale ones. An UPDATE without a WHERE clause is the most famous production incident in databases; transactions and previews are how you make sure it never happens to you.

INSERT: adding rows

We work on copies of the course tables inside our scratch database, so `ecom` stays untouched.

SQL — One row · always list the columns so the statement survives schema changes

```sql
INSERT INTO customers (id, name, email)
VALUES (401, 'maya', 'dd');
SELECT * FROM customers ORDER BY id;
```

RESULT — 4 ROWS

id	name	email
101	love	aa
201	ansh	bb
301	lamba	cc
401	maya	dd

SQL — Many rows in one statement: far faster than many single-row inserts

```sql
INSERT INTO customers (id, name, email) VALUES
       (501, 'zed',  'ee'),
       (601, 'tara', 'ff');
SELECT * FROM customers ORDER BY id;
```

RESULT — 6 ROWS

id	name	email
101	love	aa
201	ansh	bb
301	lamba	cc
401	maya	dd
501	zed	ee
601	tara	ff

SQL — Columns you skip become NULL (or their DEFAULT)

```sql
INSERT INTO customers (id, name) VALUES (701, 'no_email');
SELECT * FROM customers WHERE id = 701;
```

RESULT — 1 ROW

id	name	email
701	no_email	*NULL*

The most powerful form copies rows **from a query**: no VALUES list at all.

SQL — INSERT ... SELECT

```sql
CREATE TABLE big_orders LIKE orders;
INSERT INTO big_orders SELECT * FROM orders WHERE price >= 1100;
SELECT * FROM big_orders;
```

RESULT — 2 ROWS

o_id	cust_id	price
2	201	1100
3	501	1200

Variant	What it does with a duplicate key	Use when
`INSERT`	raises an error and stops	you expect rows to be new
`INSERT IGNORE`	silently skips the offending row (also hides other errors!)	loading data where duplicates are harmless
`REPLACE INTO`	deletes the old row, then inserts the new one (new row id, triggers fire twice)	rarely; prefer an upsert
`INSERT … ON DUPLICATE KEY UPDATE`	updates the existing row instead	idempotent loads (see below)

Upsert: the idempotent load pattern

Pipelines retry. If a job runs twice, the second run must not create duplicates. An **upsert** (*update or insert*) makes loads safe to repeat: new keys are inserted, existing keys are updated.

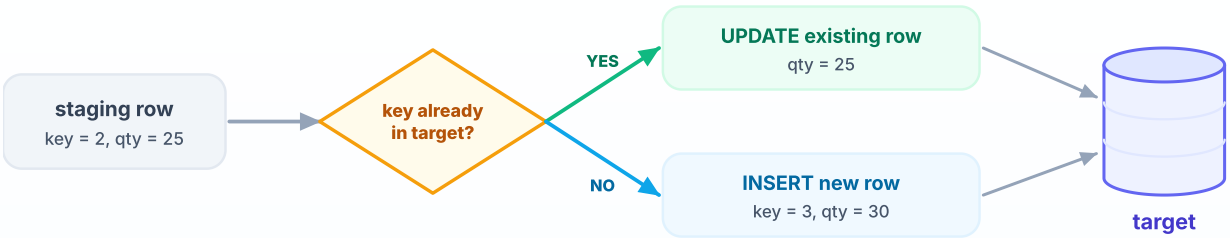


Figure 6. INSERT … ON DUPLICATE KEY UPDATE decides row by row. It needs a PRIMARY KEY or UNIQUE index to detect duplicates.

SQL — Target before the load

```sql
SELECT * FROM stock ORDER BY product_id;
```

RESULT — 2 ROWS

product_id	qty	updated_by
1	10	init
2	20	init

SQL — Staging: product 2 changed, product 3 is new

```sql
SELECT * FROM stock_staging ORDER BY product_id;
```

RESULT — 2 ROWS

product_id	qty
2	25
3	30

SQL — Upsert: product 1 untouched · 2 updated · 3 inserted

```sql
INSERT INTO stock (product_id, qty, updated_by)
SELECT product_id, qty, 'load' FROM stock_staging
ON DUPLICATE KEY UPDATE qty = VALUES(qty), updated_by = 'load';
SELECT * FROM stock ORDER BY product_id;
```

RESULT — 3 ROWS

product_id	qty	updated_by
1	10	init
2	25	load
3	30	load

MYSQL SPECIFIC

`VALUES(col)` inside `ON DUPLICATE KEY UPDATE` is deprecated from MySQL 8.0.20 in favor of a row alias: `INSERT … VALUES (…) AS new ON DUPLICATE KEY UPDATE qty = new.qty`. Both work in MySQL 8; the alias form is not available in MariaDB.

DATA-ENGINEERING LENS

Run that upsert twice: the table looks identical afterwards. That property, **idempotency**, is what lets orchestrators like Airflow retry failed tasks safely. For loading files, MySQL also offers `LOAD DATA LOCAL INFILE 'sales.csv' INTO TABLE t FIELDS TERMINATED BY ',' IGNORE 1 LINES;`, usually the fastest way to bulk-load a CSV.

UPDATE: changing existing rows

`UPDATE table SET column = value WHERE condition`. The `WHERE` clause chooses the rows; without it, **every** row changes.

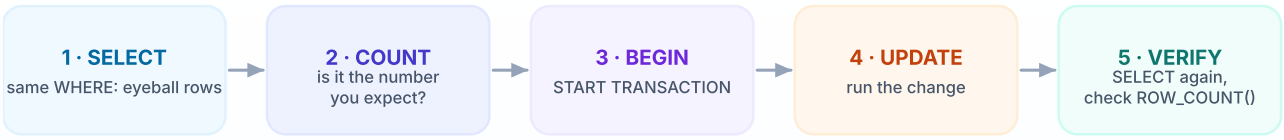
SQL — Course ch-5 UPDATE: love → sam

```sql
UPDATE customers
SET    name = 'sam'
WHERE  email = 'aa';
SELECT * FROM customers WHERE id = 101;
```

RESULT — 1 ROW

id	name	email
101	sam	aa

A safe workflow for every UPDATE or DELETE



Looks right → COMMIT. Looks wrong → ROLLBACK. Nothing is final until you say so.

Figure 7. Five steps that turn a risky statement into a reversible one.

You can update several columns, use expressions, and even update from another table with a join:

SQL — UPDATE with JOIN: +10% on the orders of one customer

```sql
UPDATE orders o
JOIN   customers c ON o.cust_id = c.id
SET    o.price = o.price * 1.10
WHERE  c.name = 'ansh';
SELECT * FROM orders ORDER BY o_id;
```

RESULT — 3 ROWS

o_id	cust_id	price
1	101	1000
2	201	1210
3	501	1200

> ⚠ **WATCH OUT**
> MySQL Workbench enables **Safe Updates** by default and refuses `UPDATE` / `DELETE` that lack a `WHERE` on a key column (Error 1175). It is a seatbelt, not a nuisance: learn to write the key-based `WHERE` instead of switching the option off.

DELETE: removing rows

SQL — Course ch-5 DELETE: the row for 'aa' is gone

```sql
DELETE FROM customers
WHERE  email = 'aa';
SELECT * FROM customers ORDER BY id;
```

RESULT — 6 ROWS

id	name	email
201	ansh	bb
301	lamba	cc
401	maya	dd
501	zed	ee
601	tara	ff
701	no_email	*NULL*

- `DELETE FROM t;` (no WHERE) removes **all** rows, but one by one and logged. For a full wipe, `TRUNCATE` is far faster (Phase 2).
- Deleting parent rows may be blocked or cascaded by foreign keys (Phase 2).
- `DELETE FROM t WHERE … LIMIT 10000;` in a loop removes huge numbers of rows in batches without locking the table for long.
- **Soft delete**: instead of removing a row, set `is_deleted = 1` or `deleted_at = NOW()`. Warehouses prefer this because history and audits stay intact; downstream models just filter `WHERE NOT is_deleted`.

Transactions: all or nothing

A **transaction** groups several statements so they succeed or fail *together*. Until you `COMMIT`, other sessions do not see your changes, and `ROLLBACK` undoes them. Here is the nightmare scenario (UPDATE without WHERE) and its rescue:

SQL — One session; two snapshots

```sql
START TRANSACTION;
UPDATE customers SET name = 'oops';            -- forgot the WHERE clause!
SELECT id, name FROM customers ORDER BY id;    -- everybody is 'oops'
ROLLBACK;
SELECT id, name FROM customers ORDER BY id;    -- as if nothing happened
```

INSIDE THE TRANSACTION — 6 ROWS

id	name
201	oops
301	oops
401	oops
501	oops
601	oops
701	oops

AFTER ROLLBACK — 6 ROWS

id	name
201	ansh
301	lamba
401	maya
501	zed
601	tara
701	no_email

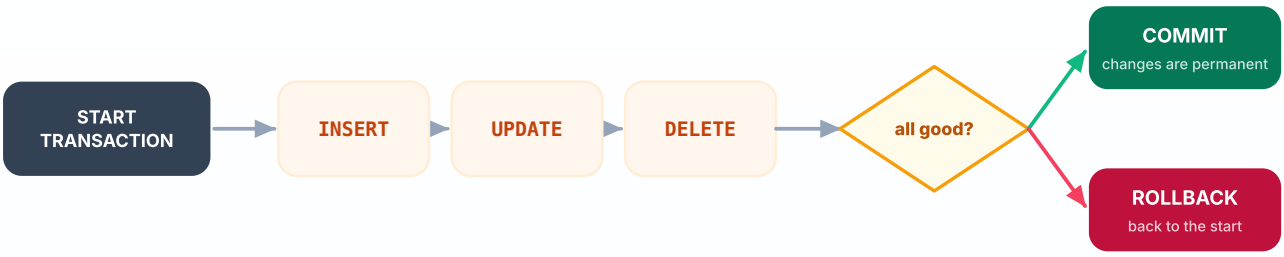


Figure 8. The life of a transaction. Nothing becomes permanent until COMMIT.

A · Atomicity

All statements apply, or none do. A half-finished transfer can never exist.

C · Consistency

Every commit leaves the data satisfying all constraints and keys.

I · Isolation

Concurrent transactions don't see each other's half-done work.

D · Durability

Once committed, data survives crashes and power loss.

MYSQL SPECIFIC

By default MySQL runs in **autocommit** mode: each statement is its own mini-transaction and is committed instantly. `START TRANSACTION` switches that off until you `COMMIT` or `ROLLBACK`. Only **InnoDB** tables support transactions (MyISAM does not), and `DDL` statements like `TRUNCATE` or `ALTER` commit implicitly and cannot be rolled back.

DATA-ENGINEERING LENS

Real pipelines wrap *delete-then-insert* of a day's partition in one transaction, so consumers either see yesterday's complete data or today's complete data, never a half-loaded table. (Cloud warehouses offer the same through `MERGE` statements and atomic table swaps.)

4.3 WRAP-UP

Phase 4 Recap & Practice

Joins and DML are best learned by doing. Complete the lab, then check your answers.

Cheat sheet

Need	Pattern
Only matching rows	`a INNER JOIN b ON a.k = b.k`
All of A, matches from B	`a LEFT JOIN b ON …` (unmatched B columns = NULL)
Rows of A with no match	`a LEFT JOIN b ON … WHERE b.k IS NULL`
FULL join in MySQL	`LEFT JOIN … UNION … RIGHT JOIN …`
Stack query results	`UNION ALL` (keep dups) · `UNION` (de-dup) · `INTERSECT` · `EXCEPT`
All combinations	`a CROSS JOIN b`
Hierarchy in one table	`emp e LEFT JOIN emp m ON e.manager_id = m.emp_id`
Insert	`INSERT INTO t (cols) VALUES (…), (…)` · `INSERT INTO t SELECT …`
Upsert	`INSERT … ON DUPLICATE KEY UPDATE col = VALUES(col)`
Update / Delete	`UPDATE t SET c = v WHERE …` · `DELETE FROM t WHERE …` (SELECT first!)
Transactions	`START TRANSACTION; … COMMIT;` or `ROLLBACK;` (InnoDB only)

Check yourself

Q1 INNER vs LEFT JOIN?

INNER keeps only rows with a match on both sides; LEFT keeps every left row and fills right columns with NULL when there is no match.

Q2 How do you emulate a FULL OUTER JOIN in MySQL?

`LEFT JOIN … UNION … RIGHT JOIN …` (UNION removes the duplicated matched rows).

Q3 Why did my LEFT JOIN behave like an INNER JOIN?

A filter on a right-table column in WHERE discards the NULL-extended rows. Put it in the ON clause.

Q4 Why is SUM() after a join too large?

The join fanned out one-to-many, repeating the parent's value per child row. Aggregate separately or join to a de-duplicated set.

Q5 What does ON DUPLICATE KEY UPDATE give you in a pipeline?

Idempotent loads: re-running the job updates existing keys and inserts new ones without creating duplicates.

Q6 What does ROLLBACK undo, and what can't it undo?

All uncommitted DML in the transaction; it can't undo committed changes or DDL such as TRUNCATE/ALTER (which auto-commit).

Practice lab

1. (`ecom`) List every order with the customer's name; include orders that have no customer.
2. (`sales`) Total revenue per store `region`, highest first.
3. (`sales`) The 5 countries (from `dim_customer`) with the highest revenue.
4. (`sales`) How many *distinct* customers bought in each product `category`?
5. (`sales`) Which products were **never sold**? (anti-join, count them)
6. (`lab`) Make a copy of `orders`, raise every price above 1000 by 5%, verify, then undo the whole thing with a transaction.

Answer key

SQL 1

```sql
SELECT o.o_id, o.price, c.name FROM orders o LEFT JOIN customers c ON o.cust_id = c.id ORDER BY o.o_id;
```

RESULT 3 ROWS

o_id	price	name
1	1000	love
2	1100	ansh
3	1200	*NULL*

SQL 2

```sql
SELECT s.region, ROUND(SUM(f.total_amount)) AS revenue
FROM   fact_sales f JOIN dim_store s ON f.store_key = s.store_key
GROUP  BY s.region ORDER BY revenue DESC;
```

RESULT 5 ROWS

region	revenue
Central	3640013
West	2871561
South	2818839
East	2476931
North	2150109

SQL 3

```sql
SELECT   c.country, ROUND(SUM(f.total_amount)) AS revenue
FROM     fact_sales f JOIN dim_customer c ON f.customer_key = c.customer_key
GROUP BY c.country
ORDER BY revenue DESC, c.country
LIMIT 5;
```

RESULT 5 ROWS

country	revenue
Korea	248790
Micronesia	197924
Fiji	195328
Norway	175845
Northern Mariana Islands	165769

SQL 4

```sql
SELECT   p.category, COUNT(DISTINCT f.customer_key) AS buyers
FROM     fact_sales f JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.category ORDER BY buyers DESC;
```

RESULT 6 ROWS

category	buyers
Home & Kitchen	430
Sports	416
Clothing	413
Books	409
Toys	396
Electronics	388

SQL 5

```sql
SELECT COUNT(*) AS never_sold
FROM   dim_product p
LEFT JOIN fact_sales f ON f.product_key = p.product_key
WHERE  f.sales_id IS NULL;
```

RESULT 1 ROW

never_sold
0

SQL 6

```sql
CREATE TABLE orders_copy AS SELECT * FROM orders;
START TRANSACTION;
UPDATE orders_copy SET price = price * 1.05 WHERE price > 1000;
SELECT * FROM orders_copy ORDER BY o_id;
ROLLBACK;
SELECT * FROM orders_copy ORDER BY o_id;
```

INSIDE THE TRANSACTION 3 ROWS

o_id	cust_id	price
1	101	1000
2	201	1271
3	501	1260

AFTER ROLLBACK 3 ROWS

o_id	cust_id	price
1	101	1000
2	201	1210
3	501	1200

COMING IN PHASE 5

Phase 5: Functions & Transformations. Learn the toolbox for cleaning and reshaping column values: numeric, string and date functions, type casting, and `CASE` logic.